

# Qué son

Una estructura de datos es una colección de valores relacionados.

Como una *lista* o un *diccionario*.

Nos permiten organizar y acceder información por medio de conexiones entre **nodos**.

Que es un **nodo**? El objeto que contiene la información que queremos guardar.

Cada tipo de estructura de datos relaciona estos **nodos** de manera distinta.

A pesar de que ya todos los lenguajes de alto nivel tienen estructuras predefinidas que funcionan casi para cualquier uso, siempre es bueno tener un entendimiento de las estructuras de datos "primitivas" y cómo funcionan.

Además, nos sirven para practicar conceptos de OOP.

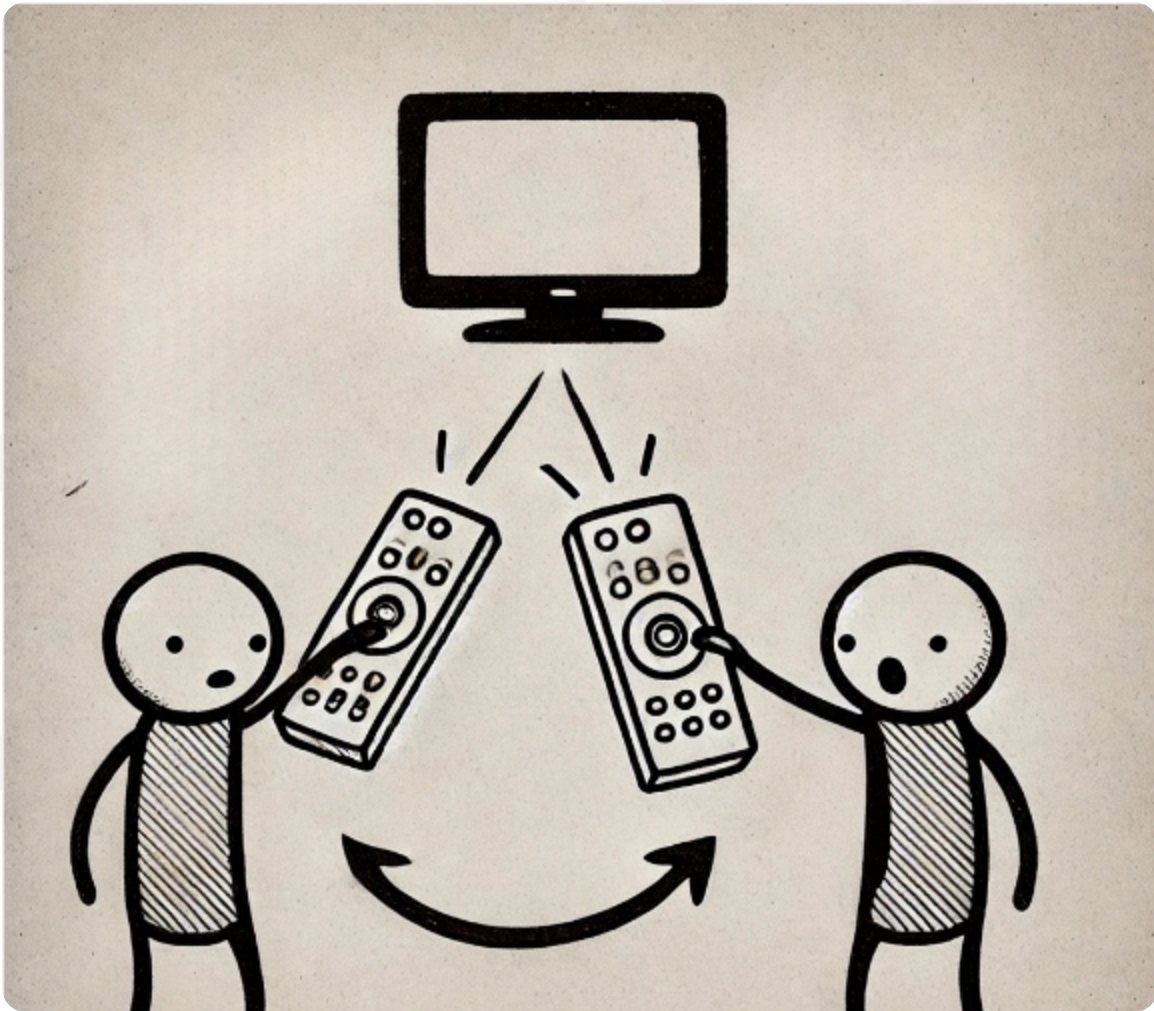
## Referencias en estructuras de datos

En programación, una **referencia** es un vínculo que apunta a la ubicación en memoria donde se almacena un objeto. Esto significa que cuando asignamos una variable a otra, en lugar de copiar el valor del objeto, simplemente estamos creando un nuevo nombre que apunta a la misma ubicación en memoria.

### Ejemplo en la vida cotidiana

Imagina que tienes un control remoto para la televisión. Si le das otro control a tu amigo y ambos están sincronizados con la misma televisión, ambos pueden cambiar los canales. No hay dos televisores, solo dos controles apuntando al mismo dispositivo.

En programación, una **referencia** funciona de la misma manera: dos variables pueden apuntar al mismo objeto en memoria, permitiendo modificarlo desde distintos lugares sin duplicarlo.



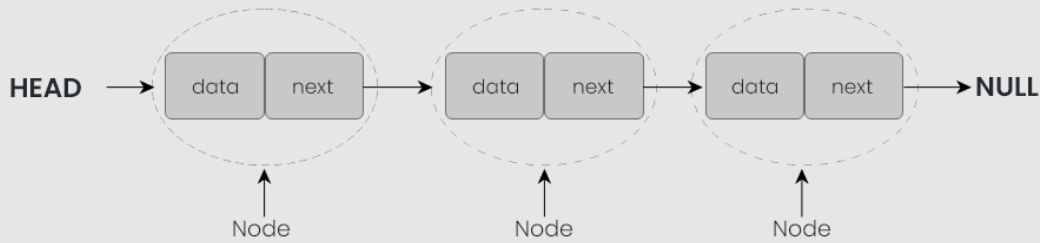
## Linked Lists

Es una estructura que tiene un atributo `head` que **referencia** a su primer elemento o `nodo`.

Cada `nodo` tiene un atributo `next` que **referencia** al siguiente `nodo` de la lista.

Ese siguiente `nodo` tiene el mismo atributo, que **referencia** al siguiente `nodo`, y así sucesivamente hasta llegar al ultimo nodo (el que no tenga un `next` es el ultimo).

## Linked List Data Structure



Veamos como se vería un Linked List en código.

Primero debemos crear la clase de la estructura.

Vamos a agregarle un constructor que acepte el `nodo` que será su `head`.

```

1 class LinkedList:
2     def __init__(self, head):
3         self.head = head
    
```

Copiar

Ahora necesitamos crear la clase de los `nodos`:

También vamos a agregarle un constructor que acepte la `data`, y el `nodo` que será su `next`.

```

1 class Node:
2     data: str
3     next: "Node"
4
5     def __init__(self, data, next=None):
6         self.data = data
7         self.next = next
8
9 class LinkedList:
10     head: Node
11
12     def __init__(self, head):
13         self.head = head
    
```

Copiar

El atributo `data` va a tener cualquier valor que queramos en el `nodo`. En este caso, crearemos una `LinkedList` de strings, así que el `data` de cada `nodo` será un string.

Perfecto, ahora podemos crear una Linked List con algunos nodos.

```
1 class Node:
2     data: str
3     next: "Node"
4
5     def __init__(self, data, next=None):
6         self.data = data
7         self.next = next
8
9 class LinkedList:
10     head: Node
11
12     def __init__(self, head):
13         self.head = head
14
15
16 third_node = Node("Soy el tercer nodo")
17 second_node = Node("Soy el segundo nodo", third_node)
18 first_node = Node("Soy el primer nodo", second_node)
19
20 linked_list = LinkedList(first_node)
```

Copiar

En este caso, el `third_node` será el último nodo.

El `second_node` será el del medio y su `next` será el `third_node`.

El `first_node` será el `head` de `linked_list` y su `next` será el `second_node`.

Ahora necesitamos una manera de printear sus nodos para saber que está funcionando correctamente.

Lo primero que necesitamos es tomar el primer nodo, y sabemos que es el `head`.

Teniendo ese `head`, podemos hacer print de su `data`.

Copiar

```
1 class Node:
2     data: str
3     next: "Node"
4
5     def __init__(self, data, next=None):
6         self.data = data
7         self.next = next
8
9 class LinkedList:
10     head: Node
11
12     def __init__(self, head):
13         self.head = head
14
15     def print_structure(self):
16         current_node = self.head
17         print(current_node.data)
18
19
20 third_node = Node("Soy el tercer nodo")
21 second_node = Node("Soy el segundo nodo", third_node)
22 first_node = Node("Soy el primer nodo", second_node)
23
24 linked_list = LinkedList(first_node)
25 linked_list.print_structure()
```



Soy el primer nodo

Perfecto, ahora si quisieramos hacer print del segundo nodo también, tenemos que accederlo usando el `next` del primero (que ya tenemos guardado en la variable `current_node`).

Copiar

```
1 class Node:
2     data: str
3     next: "Node"
4
5     def __init__(self, data, next=None):
6         self.data = data
7         self.next = next
8
9 class LinkedList:
10     head: Node
11
12     def __init__(self, head):
13         self.head = head
14
15     def print_structure(self):
16         current_node = self.head
17         print(current_node.data)
18
19         current_node = current_node.next
20         print(current_node.data)
21
22
23 third_node = Node("Soy el tercer nodo")
24 second_node = Node("Soy el segundo nodo", third_node)
25 first_node = Node("Soy el primer nodo", second_node)
26
27 linked_list = LinkedList(first_node)
28 linked_list.print_structure()
```



Soy el primer nodo

Soy el segundo nodo

Lo mismo con el tercero: lo accedamos usando el `next` del segundo, que ya tenemos guardado como `current_node`.

Copiar

```
1 class Node:
2     data: str
3     next: "Node"
4
5     def __init__(self, data, next=None):
6         self.data = data
7         self.next = next
8
9 class LinkedList:
10     head: Node
11
12     def __init__(self, head):
13         self.head = head
14
15     def print_structure(self):
16         current_node = self.head
17         print(current_node.data)
18
19         current_node = current_node.next
20         print(current_node.data)
21
22         current_node = current_node.next
23         print(current_node.data)
24
25
26 third_node = Node("Soy el tercer nodo")
27 second_node = Node("Soy el segundo nodo", third_node)
28 first_node = Node("Soy el primer nodo", second_node)
29
30 linked_list = LinkedList(first_node)
31 linked_list.print_structure()
```



Soy el primer nodo  
Soy el segundo nodo  
Soy el tercer nodo

A pesar de que esto va muy bien, no podemos seguir haciendolo con código estático.

No funcionaría con estructuras de distintas cantidades de nodos.

Sin embargo, si ponemos atención, veremos que ya tenemos un patrón que podemos enciclar.

Copiar

```
1 class Node:
2     data: str
3     next: "Node"
4
5     def __init__(self, data, next=None):
6         self.data = data
7         self.next = next
8
9 class LinkedList:
10     head: Node
11
12     def __init__(self, head):
13         self.head = head
14
15     def print_structure(self):
16         current_node = self.head
17         print(current_node.data)
18
19         current_node = current_node.next
20         print(current_node.data)
21
22         current_node = current_node.next
23         print(current_node.data)
24
25
26 third_node = Node("Soy el tercer nodo")
27 second_node = Node("Soy el segundo nodo", third_node)
28 first_node = Node("Soy el primer nodo", second_node)
29
30 linked_list = LinkedList(first_node)
31 linked_list.print_structure()
```

Lo que tenemos que hacer es seguir sobrescribiendo el valor del `current_node` por su `next` y printeando su `data`, siempre y cuando este no sea `None`.

Si llegamos al `None`, significa que ya pasamos el ultimo nodo de la lista, cuyo `next` era ese `None`.

Esto lo podemos lograr con un while (ya que no tenemos certeza de cuantas veces debe correr, pero sí de cuando tiene que parar).

Copiar

```
1 class Node:
2     data: str
3     next: "Node"
4
5     def __init__(self, data, next=None):
6         self.data = data
7         self.next = next
8
9 class LinkedList:
10     head: Node
11
12     def __init__(self, head):
13         self.head = head
14
15     def print_structure(self):
16         current_node = self.head
17
18         while (current_node is not None):
19             print(current_node.data)
20             current_node = current_node.next
21
22
23 third_node = Node("Soy el tercer nodo")
24 second_node = Node("Soy el segundo nodo", third_node)
25 first_node = Node("Soy el primer nodo", second_node)
26
27 linked_list = LinkedList(first_node)
28 linked_list.print_structure()
```



Soy el primer nodo  
Soy el segundo nodo  
Soy el tercer nodo

## Queue (Fila)

Se basa en las Linked Lists.

Tiene un nodo `head`.

**FIFO**: First In, First Out

El primer `nodo` que entre es el primero que sale.

Como cuando uno quiere pedir algo de Mc y tiene que hacer fila. Debe esperarse a que todos los que llegaron antes ordenen primero.

Tiene dos métodos:

`enqueue` para agregar `nodos` (al final).

`dequeue` para quitar `nodos` (del inicio).

## Stack (Pila)

Se basa en las Linked Lists.

Tiene un nodo `top`.

**LIFO**: Last In, First Out

El ultimo `nodo` que entre es el primero que sale.

Como cuando uno tiene una pila de discos y quiere sacar uno que está en medio. Debe sacar todos los que puso después de ese para poder sacarlo.

Tiene dos métodos:

`push` para agregar `nodos` (al inicio).

`pop` para quitar `nodos` (del inicio).

## Double Ended Queue

Combina los `Queues` y los `Stacks` en una sola estructura.

Tiene un nodo `head` y un nodo `tail`.

Permite sacar y agregar nodos tanto al inicio como al final.

Como cuando uno tiene una bolsa de tortillas y puede tomar tortillas de arriba y de abajo.

Como una hilera de huevos.

Tiene 4 métodos: `push_right`, `push_left`, `pop_right` y `pop_left`.

# Binary Tree

Se basa en los `Linked Lists` con la diferencia de que cada nodo tiene 2 hijos.

Tiene un nodo `root`.

## Ejercicios (Pair Programing)

1. Cree una estructura de objetos que asemeje una Queue.
  1. Debe incluir los métodos de `enqueue` (para agregar nodos) y `dequeue` (para quitar nodos).
  2. Debe incluir un método para hacer `print` de toda la estructura.
  3. No se permite el uso de tipos de datos compuestos como `lists`, `dicts` o `tuples` ni módulos como `collections`.

Resolución

Copiar

```
1 class Node:
2     data: str
3
4     def __init__(self, data, next=None):
5         self.data = data
6         self.next = next
7
8 class LinkedList:
9     head: Node
10
11     def __init__(self, head):
12         self.head = head
13
14     def print_structure(self):
15         current_node = self.head
16         while (current_node is not None):
17             print(current_node.data)
18             current_node = current_node.next
19
20 class Queue(LinkedList):
21     def enqueue(self, new_node):
22         current_node = self.head
23         next_node = current_node.next
24         while (next_node is not None):
25             current_node = next_node
26             next_node = current_node.next
27
28         current_node.next = new_node
29
30     def dequeue(self):
31         self.head = self.head.next
32
33
34 tercer_nodo = Node("Soy el tercer nodo")
35 segundo_nodo = Node("Soy el segundo nodo", tercer_nodo)
36 primer_nodo = Node("Soy el primer nodo", segundo_nodo)
37
38 queue = Queue(primer_nodo)
39
```

```
40 print("Agregando un elemento")
41
42 queue.enqueue(Node("Soy el nuevo nodo!"))
43 queue.print_structure()
44
45 print("Quitando un elemento")
46
47 queue.dequeue()
48 queue.print_structure()
```